Durable Records: One Presence and Survival Law for the Local Stores of a Rings Node

Ryan J. Kung

Abstract

A Rings node keeps security state (transaction replay windows), privacy state (onion entry guards) and custody state (relay mail) in local key-value stores on two platform families: a native file system and a browser's IndexedDB. Five open defects of these stores reduce to two predicates: *presence*, whether a record that should exist exists, and *survival*, whether a completed write outlives a crash. We give one model for both. A store's authority is a point of the product of a crash chain $\mathsf{Vol} \sqsubset \mathsf{Cch} \sqsubset \mathsf{Dur}$ and a meaning chain $\mathsf{Reb} \sqsubset \mathsf{Ret} \sqsubset \mathsf{Aut}$; each point fixes what an absent record means and which obligations the platform must discharge. We choose a per-record presence marker as the witness that separates a lost authoritative record from a never-written one, prove it sound and complete for partial loss, and prove that no witness in the same failure domain detects total loss. We state the crash obligation per platform, verified against the Chromium, Gecko and WebKit sources (only `strict` durability flushes in all three), forbid eviction for authoritative records, and give a custody law that makes relay mail survive a restart between admission and consumption, with the impossibility result that bounds what any receiver can do for mail that has fallen behind live traffic. A browser can only claim durability, since an ephemeral context is indistinguishable from a persistent one; we admit a node's own state on the claim, with each ephemeral session end as the documented reset, and admit custody of other nodes' mail on a claim only across $r \geq 2$ holders.

I. Problem

The defects, each named by its issue:

#915
An absent native replay record is indistinguishable from a stream never seen, so its stream restarts from First; on non-unix targets a completed rename can roll back.

#923
The browser replay store evicts its least recently used row at its row cap, so a retained stream can restart from First.

#912
Browser replay writes use the default durability hint, so a completed admission can roll back after an OS crash.

#911
The entry-guard store is disposable: a crash can lose or tear the guard set, re-selecting the first hop.

#936
Relay mail admitted into the replay window and queued, but not taken before a restart, is refused as a replay on redelivery and lost.

#915, #923 are presence defects; #912, #911 are survival defects; #936 is a presence defect of a fact that lives in no store at all (that the queued mail was never consumed). Section II fixes the model, Section III the authority lattice, Sections IV–VIII the five laws, and Section XII the order of implementation. One defect was found after the batch:

#1036
A browser context whose IndexedDB opens but is ephemeral (a private window, ephemeral partitioned storage) claims Dur, and nothing the page observes tells it apart; Section X rules on it.

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created October 23, 2026. This document specifies the durability model of the local stores of Rings before 1.0 (`rings-core`, modules `storage`, `message::replay` and `swarm::inbox`; `rings-node`, module `onion::entry_guard`). It is the law for issues #911, #912, #915, #916, #923 and #936 of `RingsNetwork/rings`.

II. Model

Definition 1 (Store)**.** *A store is a finite partial map $\sigma : \mathcal{N} \rightharpoonup \mathcal{V}$ from record names to values. Its operations are* $\mathrm{put}(n, v)$*,* $\mathrm{rm}(n)$ *and* $\mathrm{get}(n)$*, each returning* Ok *or* Err*;* scan *returns one classification per name (Section IV). For* `FileStorage` *the name of key k is* $\mathrm{name}(k) = \mathrm{hex}(\mathrm{SHA1}(k))$*; for* `IdbStorage` *it is k.*

Definition 2 (History, completion)**.** *A history is $h = o_1 \cdots o_k \, o^{?}_{k+1}$: operations $o_1..o_k$ returned, o_{k+1} may be in flight. Let $\sigma_i = o_i \circ \cdots \circ o_1(\sigma_0)$, with $\sigma_{k+1} = \sigma_k$ when nothing is in flight. An operation is* completed *iff it returned* Ok*.*

Definition 3 (Crash, recovery image)**.** *A crash ends the process (*process crash*) or the operating system (*system crash*: kernel panic, power loss). The recovery image $R_P(h)$ is the set of maps the next open of the store can read on platform P after a crash at the end of h.*

Definition 4 (Loss events)**.** *A loss event is a change of the store's physical state that no operation of the store made:*

- $\lambda_{\mathrm{part}}(S)$*,* partial loss*: the records of $S \subsetneq \mathrm{dom}\,\sigma$ vanish (media error repaired by* `fsck`*, an operator removing or renaming one file, a completed rename rolled back by a platform that does not discharge Obligation 2);*

- λ_{tot}, total loss*: the whole failure domain vanishes (the store's directory, an IndexedDB database or origin, a process heap).*

Loss events are what the platform or the operator does, not what an adversary forges: a party that can write the store can also write any witness of it, so nothing below authenticates a store, and nothing models a state the program and platform cannot reach.

A. Obligations

Obligation 1 (Atomicity, A)**.** $\forall \sigma' \in R_P(h).\ \forall n.\ \sigma'(n) \in \{\sigma_i(n) : i \le k+1\} \cup \{\bot\}$*: a record is never torn.*

Obligation 2 (Survival, D)**.** $\forall \sigma' \in R_P(h).\ \forall n.\ \sigma'(n) \in \{\sigma_k(n), \sigma_{k+1}(n)\}$*: every completed write and removal survives a process or system crash, and the one in flight is either whole or absent. D implies A.*

Obligation 3 (No eviction, N)**.** $\text{put}(n, v)$ *on* σ *yields* $\sigma[n \mapsto v]$ *or returns* $\mathsf{Err}(\mathsf{BudgetExhausted})$ *with* σ *unchanged; no operation removes a name it was not given.*

Obligation 4 (Witnessed presence, W)**.** *After any history and any partial loss* $\lambda_{\text{part}}(S)$*,* scan *reports every* $n \in S \cap \text{dom}\,\sigma_k$ *as* $\mathsf{Missing}(n)$*, and reports* $\mathsf{Missing}(n)$ *only for an* n *whose record some* put *made durable.*

Obligation 5 (Fail closed, F)**.** *A record that does not decode is reported and kept; only its owner or an operator removes it.*

III. The authority lattice

Definition 5 (Crash chain)**.** $C = \mathsf{Vol} \sqsubset \mathsf{Cch} \sqsubset \mathsf{Dur}$ *with obligations* $O(\mathsf{Vol}) = \emptyset$ *(*$R_P(h) = \{\emptyset\}$ *at every restart),* $O(\mathsf{Cch}) = \emptyset$ *(any* $\sigma_i(n)$*,* $\bot$ *or a torn record),* $O(\mathsf{Dur}) = \{A, D\}$.

Definition 6 (Meaning chain)**.** $M = \mathsf{Reb} \sqsubset \mathsf{Ret} \sqsubset \mathsf{Aut}$*, read as what* $\sigma'(n) = \bot$ *means to the owner:*

- Reb*:* $\bot$ *is the owner's default; any record may be dropped;*
- Ret*:* $\bot$ *is "never written, or retired by the owner"; a loss is a liveness failure;*
- Aut*:* $\bot$ *is "never written"; a loss is a safety failure.*

$O(\mathsf{Reb}) = \emptyset$, $O(\mathsf{Ret}) = \{N\}$, $O(\mathsf{Aut}) = \{N, W, F\}$.

Definition 7 (Authority)**.** *An authority is* $a = (c, m) \in C \times M$*, ordered componentwise, with* $O(c, m) = O(c) \cup O(m)$*.* $C \times M$ *is a lattice (a product of chains, meet and join componentwise) and* O *is monotone:* $a \sqsubseteq b \Rightarrow O(a) \subseteq O(b)$.

Law 1 (Admissibility)**.** *Let* $\text{req}(\mathsf{Reb}) = \mathsf{Vol}$ *and* $\text{req}(\mathsf{Ret}) = \text{req}(\mathsf{Aut}) = \mathsf{Dur}$*. A store may be opened at* (c, m) *only if* $c \sqsupseteq \text{req}(m)$.

Justification. Under $c \sqsubset \mathsf{Dur}$ a crash may lose a completed write, so $\bot$ after recovery can be a loss; a meaning above Reb asserts that $\bot$ is not a loss. □

Law 1 reads c as the crash class the platform establishes. A browser can only claim it (Assumption 2); Section X states when a claimed class is admissible.

Table I
STORES OF A NODE AND THE AUTHORITY THE LAW ASSIGNS

Store	Today	Law
DHT data (native, browser)	(Cch, Reb)	(Cch, Reb)
relay carriers in it	(Cch, Reb)	(Dur, Ret) #1021
Measure, evidence	(Cch, Reb)	(Cch, Reb)
Entry guards	(Cch, Reb)	(Dur, Reb) #911
Replay (native)	(Dur, Aut)*	(Dur, Aut) #915
Replay (browser)	(Cch, Reb)†	(Dur, Aut) #912,#923
Replay (no store given)	(Vol, Aut)	refused #1022
Browser, where Dur is claimed (Dur°):		
relay carriers	(Dur, Ret)	same, $r \ge 2$ #777
replay	(Dur, Aut)	same #1036
entry guards	(Dur, Reb)	same

*W missing; D only on unix. †Decodes as Aut (F) but neither D (relaxed commit) nor N (row-cap eviction) holds.
°Claimed (Assumption 2). r: the holders of a carrier (Law 13), one today.

The existing `RecordAuthority` is the two-point chain $\mathsf{Disposable} = (\mathsf{Cch}, \mathsf{Reb}) \sqsubset \mathsf{Authoritative} = (\mathsf{Dur}, \mathsf{Aut})$ (#909, #910, #914). The law adds two points: $\mathsf{DurableCache} = (\mathsf{Dur}, \mathsf{Reb})$ for state whose loss is a downgrade, and $\mathsf{Retained} = (\mathsf{Dur}, \mathsf{Ret})$ for custody. $\mathsf{MemStorage}$ is $(\mathsf{Vol}, -)$: admissible only at Reb.

Table I is the whole classification. The two rows marked with new issues are inadmissible points the law exposes outside the five issues: relay carriers are the only copy of held mail (#1021), and a provider given no replay store runs replay at (Vol, Aut) (#1022). The browser rows below the rule rest on a claimed crash class (Section X); the carrier row is inadmissible at $r = 1$, which #777 closes.

IV. Presence witness

A. Choice

W needs, at restore, a set $\text{wit} \subseteq \mathcal{N}$ of names that were durably written, so that $\text{wit} \setminus \text{dom}\,\sigma'$ are the lost names. Fail-closed per stream (#910) is keyed by record name: $\text{unavailable}(s) \iff \text{name}(\text{rec}(s)) \in \mathcal{U}$. A witness that does not name the lost record cannot feed $\mathcal{U}$. The candidates:

Witness	names loss	$O(1)$/new key	operator step
count $\lvert\text{dom}\,\sigma\rvert$	no	yes	edit count
epoch per record	no	yes	edit epoch
manifest log	yes	yes	edit binary log
per-record marker	yes	yes	remove a file

A count or an epoch detects that something is missing but not which stream, so it can only fail the whole store closed: one lost file would close every stream. A manifest names the loss but turns the operator's per-stream recovery into editing a binary log, and its torn tail needs its own law. We choose the marker.

Definition 8 (Presence marker)**.** *For an authoritative native store, the marker of name* n *is an empty file* n.p *in the store's own directory. Let* R *be the set of record names listed,* P *the set of names with a marker, both from one listing.*

Law 2 (Markers are not records)**.** *A name ending in* .p *is never a record name: the index and* `entry_file_name` *skip it, so it is never indexed, scanned as* Undecodable*,*

counted against the byte budget, or counted by `TRANSACTION_REPLAY_STORE_MAX_RECORDS` *or the refusal of Law 8. Every bound counts R only.*

Law 3 (Write order)**.** *An authoritative* $\mathrm{put}(n, v)$ *runs*

$$\mathrm{write}(n.\mathtt{tmp});\ \mathrm{sync};\ \mathrm{rename}(n.\mathtt{tmp}, n);\ \mathrm{sync}_{\mathrm{dir}};$$
$$[\, n \notin P :\ \mathrm{create}(n.\mathtt{p});\ \mathrm{sync}_{\mathrm{dir}}\,]$$

and returns Ok *only after the last step. An authoritative* $\mathrm{rm}(n)$ *runs* $\mathrm{remove}(n.\mathtt{p}); \mathrm{sync}_{\mathrm{dir}}; \mathrm{remove}(n); \mathrm{sync}_{\mathrm{dir}}$.

Invariant 1 (Marker implies record)**.** *In every physical state the program and a crash reach, $P \subseteq R$.*

Proof. A marker is created only after the rename that makes $n \in R$ is flushed, and removed before the record. A crash between any two steps of Law 3 leaves either P unchanged or the record already durable; by D the flushed prefix is what recovery sees. □

Law 4 (Restore)**.** *Restore classifies, from one listing:*

$$\begin{aligned} n \in R,\ \textit{decodes as its key} &\ \mapsto\ \mathsf{Filed}(k, v),\\ n \in R,\ \textit{otherwise} &\ \mapsto\ \mathsf{Undecodable}(n),\\ n \in P \setminus R &\ \mapsto\ \mathsf{Missing}(n), \end{aligned}$$

and for $n \in R \setminus P$ creates $n.\mathtt{p}$ (flushed) before the store serves its first operation. The owner's $\mathcal{U}$ is $\{n : \mathsf{Undecodable}(n)\} \cup \{n : \mathsf{Missing}(n)\}$.

$R \setminus P$ is the crash between the rename and the marker: the record is whole and present, so it witnesses itself, and the repair restores $P = R$.

A rolled back rename is not among them: under D a completed rename does not roll back, and on a platform without D the marker's creation is not ordered after the rename, so both may vanish together. It stays a loss event of Definition 4, which Law 6 removes on Windows.

Proposition 1 (W holds)**.** *On a platform that discharges D, Laws 3 and 4 discharge Obligation 4 for every partial loss that leaves markers: a removed or renamed record file, or a record lost by* `fsck`*.*

Proof. Complete. Let $n \in \mathrm{dom}\,\sigma_k$; its first put completed, so $n \in P$ (Law 3). If n is lost and its marker is not, $n \in P \setminus R$, reported Missing. *Sound.* $\mathsf{Missing}(n)$ requires $n \in P$, and a marker is created only after a put of n flushed its rename (Law 3), so that record was durable. A crash before the marker's creation leaves $n \in R \setminus P$, never Missing: a put that never reached its rename is never reported lost. □

Proposition 2 (Total loss is undetectable)**.** *No witness stored in the failure domain of the records detects* λ_{tot}.

Proof. After λ_{tot} the physical state equals that of a store never opened; any function of the physical state takes the same value on both. □

Total loss is therefore the documented reset: it is what the operator does by removing the replay store, and its consequences are those of clearing every stream (the recovery law of `message::replay`): a receiver may admit an unexpired transaction once more, and a sender restarts at sequence 0, which its destinations see as Stale or Fork until it passes their retained high watermark. A witness outside the domain (a sibling file in the data directory) would detect a vanished store directory, but the only producer of that state is the operator, whose clearing step it would then have to name; we do not add it.

Law 5 (Read agreement)**.** *An authoritative* $\mathrm{get}(k)$ *for* $\mathrm{name}(k) \in P$ *whose record is not readable as the record of k returns* Err*, never* $\mathsf{Ok}(\bot)$*;* $\mathsf{Ok}(\bot)$ *means* $\mathrm{name}(k) \notin P$.

This closes the two cases added to #915 in review: a store whose root vanishes while the node runs answers Err for every witnessed name (as its persist already fails), and an upper-cased record file is never absent: its name is no record name, so it is never indexed, and its lower-case marker keeps it in the scan. On a case-sensitive volume it is Missing to scan, and so in $\mathcal{U}$; on a case-insensitive one the read of the marked lower-case name opens the file, so scan files it with its intact state, and get returns its value too. Either way the stream keeps its guarantee: failed closed in the first case, restored whole in the second.

Assumption 1 (Browsers lose whole origins)**.** *A browser deletes stored data per origin, never part of it [1]; this includes storage-pressure eviction of a non-persistent origin and WebKit's deletion of script-written data after seven days without user interaction.*

Proposition 3 (IndexedDB needs no marker)**.** *Under Assumption 1 and N, an IndexedDB store discharges W with $P = \emptyset$.*

Proof. Every IndexedDB write is a transaction, atomic by the specification [2]; with N the program removes no row it was not told to. By Assumption 1 the platform loses no proper subset of a database's rows, so no λ_{part} is reachable and W holds vacuously; total loss is Proposition 2. □

Assumption 2 (A browser's crash class is claimed)**.** *A page cannot observe whether its IndexedDB outlives the browsing session. An ephemeral context (a private window, ephemeral partitioned third-party storage) opens, commits and fires* `complete` *as a persistent one does, and* `navigator.storage.persisted()` *reports exemption from eviction under storage pressure, not ephemerality [1]. We write* Dur° *for a claimed* Dur*:* Dur *in a persistent context, and* Vol *with a* λ_{tot} *at session end in an ephemeral one.*

Assumption 1 holds in an ephemeral context too: its session end loses the whole partition, a total loss, so Proposition 3 is unchanged. By Proposition 2 no witness in the domain detects that loss, so no observation at open separates Dur° from Dur.

Cost. W adds one marker creation and one directory flush to the first put of each name, and nothing to later puts; restore lists one directory, as today. The replay store creates a name once per stream, so the cost is per new stream, not per transition.

Operator step. To clear one failed stream, stop the node and remove both `<name>` and `<name>.p` from `transaction-replay`. A store written before W has

$P = \emptyset$ and is repaired by Law 4 at its first open: no migration is needed.

V. Survival per platform

D is a property of the platform's commit, so each platform discharges it separately.

Unix. Law 3 with sync = `File :: sync_all`, which the Rust standard library maps to `fcntl(F_FULLFSYNC)` on Apple targets and to `fsync` elsewhere [3], and $\mathrm{sync}_{\mathrm{dir}}$ the same on the directory. The store's own directory entry is flushed at open, with its ancestors. The rename-and-sync pattern is the one crash-consistency studies find portable across unix file systems [4]. `test_durability` model-checks the step plans over every crash point.

Windows. A directory cannot be opened for flushing through the standard library, so $\mathrm{sync}_{\mathrm{dir}}$ is a no-op today and a completed rename can roll back (#915). The windows gateway is built and run in CI, so the target is in scope.

Law 6 (Write-through rename)**.** *On Windows every directory mutation of a* Dur *store is a rename by* `MoveFileExW` *with* `MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH`*, which does not return until the move is on disk [5]: a put renames its flushed temporary file; a marker is created as a flushed temporary file renamed into place; a removal renames its file to* `.tmp` *and then deletes it.*

A removal's deletion need not be durable: a `.tmp` left by a crash is removed at the next open (the index law of `FileStorage`), and it is not a record, so it never reappears.

IndexedDB. The specification makes the durability hint advisory [2]. What each engine does, read from its source:

Engine	`default`	`strict`
Chromium	relaxed since 121: complete fires once the data reaches the OS buffer [6]	flush before complete
Gecko	SQLite WAL, synchronous `NORMAL` (`FULL` under a pref) [7]	synchronous `EXTRA`
WebKit	SQLite WAL, `synchronous` left at the library default [8]	a `FULL` WAL checkpoint after commit

In WAL mode a commit under `synchronous=NORMAL` survives a process crash but may roll back after a system crash [9], which is Chromium's relaxed behaviour too. So `default` discharges D against a process crash in all three engines, and against a system crash in none that we can rely on.

Law 7 (Strict commit)**.** *Every read-write transaction of an IndexedDB store at* $c =$ Dur *is opened with* $\{$*durability* : `strict`$\}$*, and the store reports* Ok *only on the transaction's* `complete` *event.*

Under Law 7 D holds to the strength of each engine's strongest commit. We claim no more: whether that commit also flushes a drive's write cache is the engine's choice, as `F_FULLFSYNC` is ours on native Apple targets. `rexie` 0.4 opens no transaction with options; the binding is `IdbDatabase::transaction_with_str_sequence_and_mode_and_options`, which `web-sys` gates behind `web_sys_unstable_apis`, so the implementation either enables that cfg for the core crate or passes the options object through `js-sys`.

Corollary 1 (Authoritative reads need no write)**.** *`IdbStorage` restamps a row on every* get *to order eviction. Under N eviction never runs, so an authoritative* get *may be a read-only transaction.*

VI. No eviction

Law 8 (Refuse, never evict)**.** *For* $m \sqsupseteq$ Ret *on any backend: a* put *of a new name into a full store returns* Err(BudgetExhausted) *with the store unchanged; rewriting an existing name never needs room; an open with a capacity below the stored count fails.*

Proposition 4 (Eviction is a forged absence)**.** *If an authoritative IndexedDB store evicts, W fails.*

Proof. An evicted row $n \in \mathrm{dom}\,\sigma_k$ is gone with no marker (Proposition 3 has $P = \emptyset$), so restore reports nothing for it and its stream restarts from First: exactly #923. □

With Law 8 the browser replay store matches the native one (#914): a new stream past the bound fails closed, and the row capacity `TRANSACTION_REPLAY_STORE_MAX_RECORDS` becomes a refusal bound rather than a sizing hope. Whether a remote peer can drive a browser node to the cap stops mattering to safety; reaching it costs only new streams.

VII. Entry guards

Law 9 (Guards are a durable cache)**.** *The entry-guard store is opened at* (Dur, Reb)*,* `DurableCache`*: writes discharge D (native: Law 3 without markers, Law 6; browser: Law 7); a guard record that is absent or does not decode is retired and the set is re-selected from scratch, logged at* `warn` *with the record's name.*

Proposition 5 (Guards change only by reconciliation)**.** *Across a crash, the guard set the next selection reads is the last one written, unless a loss event (Definition 4) removed it.*

Proof. D gives $\sigma'(n) \in \{\sigma_k(n), \sigma_{k+1}(n)\}$ for the one record per network; both are whole sets that `reconcile_guard_state` itself produced. □

The meaning stays Reb, which answers the policy question of #911: guards are privacy state whose loss is a downgrade (one re-selection of the first hop), not a safety failure, so failing onion routing closed on an undecodable record (F) would trade availability for nothing the guard set protects. Under D an undecodable guard record is no longer reachable by a crash, only by a loss event or a schema change, which the operator cleans before 1.0. The browser fallback to an in-memory guard store when IndexedDB cannot open is (Vol, Reb), admissible, and already logged.

VIII. Relay custody

A. Model

A relay element e carries a signed transaction of stream s (origin account, this node, class) at sequence q with digest h; write $e = (s, q, h)$. The actors:

- the carrier $\mathrm{stored}(d)$: the elements held for d at $d{+}1$, retired only by d's covering remove up to an element's dot;
- the receiver window $\mathcal{W}_s$, persisted at $(\mathsf{Dur}, \mathsf{Aut})$, with the pure transition $\mathrm{observe}(\mathcal{W}_s, q, h) \in \{\mathsf{First}, \mathsf{Advance}, \mathsf{Late}, \mathsf{Replay}, \mathsf{Fork}, \mathsf{Stale}\}$;
- the application inbox, volatile; *taking* a message is where core's custody ends.

Today a drain admits h into $\mathcal{W}_s$ and queues the message; a restart before the take loses the queue but not $\mathcal{W}_s$, so the redelivery observes Replay and is retired (#936). The fact that is lost is "h was admitted for e and not taken".

Definition 9 (Custody set)**.** *The custody set of stream s is $\mathcal{K}_s \subseteq \mathbb{N} \times \mathrm{Digest}$: the relay deliveries of s admitted and not yet taken. The receiver record of s is the pair $(\mathcal{W}_s, \mathcal{K}_s)$, one record under the existing key, so one write commits both atomically (A on both platforms).*

Placing $\mathcal{K}_s$ in the receiver record, and not in the application or a separate store, is forced by two constraints: core never consults the application (the layering law), and a second record could not be committed atomically with $\mathcal{W}_s$, which would reopen the window between admission and its custody.

Law 10 (Custody)**.** *A drain pass first restricts custody to the carrier, $\mathcal{K}_s \leftarrow \mathcal{K}_s \cap \{(q,h) : (s,q,h) \in \mathrm{stored}(d)\}$ (written only if it changed), then offers each element $e = (s,q,h)$ it has not handed over in this process by the first arm that applies:*

1) *$(q,h) \in \mathcal{K}_s$:* Requeue*;*
2) *$|\bigcup_s \mathcal{K}_s| =$ `RELAY_INBOX_MAX_LEN`:* Defer*;*
3) *$\mathrm{observe}(\mathcal{W}_s, q, h) = (\mathcal{W}'_s, v)$ with v permitting dispatch: commit $(\mathcal{W}'_s, \mathcal{K}_s \cup \{(q,h)\})$, then queue;*
4) *otherwise:* Retire*,*

where Requeue *queues the message again without touching $\mathcal{W}_s$, and every queueing reserves room first (*Defer *when there is none, as today). When the application takes a message of e, core commits $(\mathcal{W}_s, \mathcal{K}_s \setminus \{(q,h)\})$, the* release*; the element is retired from the carrier only after its release committed. The live path is unchanged: it observes $\mathcal{W}_s$ alone.*

Lemma 1 (Bounded custody)**.** $|\bigcup_s \mathcal{K}_s| \le$ `RELAY_INBOX_MAX_LEN` (64)*, so a receiver record grows by at most 64 entries of $8 + 32$ bytes over the whole store.*

Proof. The Defer arm refuses a new entry at the bound, and the restriction step keeps only entries of elements still stored, of which a carrier holds at most 64. □

Proposition 6 (At least once across restart)**.** *Let the carrier satisfy $(\mathsf{Dur}, \mathsf{Ret})$ (#1021). In every history with any number of crashes, an element $e = (s,q,h)$ leaves the carrier only if (i) the application took a message of e, (ii) the live path would drop h too, or (iii) its retention bound elapsed.*

Proof. The drain retires e in two arms. After a release: the release follows a take, case (i). In the last arm of Law 10: $(q,h) \notin \mathcal{K}_s$ and observe refuses. If observe is Replay, h is in $\mathcal{W}_s$, so it was admitted earlier either live (dispatched to the application, case (ii)) or by a relay delivery, which entered $\mathcal{K}_s$ in the same commit and left it only by a release (case (i)) or by the restriction step, which requires $e \notin \mathrm{stored}(d)$. Fork and Stale are refusals of the live path, case (ii). A crash cannot separate the admission from its custody entry, nor a release from its effect, because each is one record write under D. Case (iii) is the retention law of #913. □

Proposition 7 (Duplicate window)**.** *The application receives a message of one element twice iff a crash falls after its take and before its release committed, and then at most once more per such crash.*

Proof. Before the release, $(q,h) \in \mathcal{K}_s$ survives the crash and the redelivery is a Requeue. After it, $(q,h) \notin \mathcal{K}_s$ and $h \in \mathcal{W}_s$ or below its window, so the redelivery is refused and retired. □

The duplicate carries the same transaction id, so an application that needs exactly-once delivery deduplicates by it. Moving the release before the take returns would make the inbox's synchronous `try_next` perform a store write; we keep the take synchronous and accept the window, which is one record write long when the release is issued on the take's consumption signal.

B. Mail behind live traffic

#936 also asks that a relay message behind live traffic of the same origin be delivered. Inside the window it is: an empty slot gives Late. Below it, no receiver can.

Proposition 8 (No bounded receiver admits stale mail safely)**.** *Let $q < \mathrm{rmin}(\mathcal{W}_s)$ and $(q,h) \notin \mathcal{K}_s$. No decision that depends only on bounded receiver state both admits every such e whose h was never dispatched and refuses every such e whose h was.*

Proof. Take two histories that end in the same $(\mathcal{W}_s, \mathcal{K}_s)$: in the first, h arrived live, was dispatched, and at least `REORDER_WINDOW` later sequences followed; in the second, h was held, never delivered, and the same later sequences arrived live. A holder may hold a payload it forwarded (any hold inside the sender's proof lifetime is admissible), so e can be stored in both. The window retains only the last 32 digests, so the states agree, and any function of them decides both alike: admitting is a replay in the first, refusing is a loss in the second. Separating them needs every digest admitted within the sender's proof lifetime, which no bound on the store's size covers. □

So below the window the drain refuses as the live path does, which is the criterion #936 itself states (a refusal retires an element only when core would drop the same payload arriving live). Delivering such mail needs the sender to put it on a stream that live traffic does not advance, a protocol change outside this law.

IX. Group commit

One lock serializes every replay transition, and under Law 3 or Law 7 each one paid a flush: about 55 transitions per second on the native store (macOS) and about 65 in a browser (#916). Group commit keeps the lock and the flush, and moves the wait from the lock to the release.

Definition 10 (Transition, batch)**.** *Transitions $t_1 < t_2 < \cdots$ run in lock order on the* speculative *tables $\hat{T}$: $t_i(\hat{T}_{i-1}) = (\hat{T}_i, w_i, r_i)$ with an optional record write $w_i = (n_i, v_i)$, a result r_i, and an undo entry u_i with $u_i(\hat{T}_i) = \hat{T}_{i-1}$. Batches $B_1, B_2, \ldots$ partition the transitions into intervals of lock order. Batch B_j is written by one* $\mathrm{put}^*(W_j)$*, where $W_j(n)$ is the last v_i of B_j written under n; the batches are written one at a time, in order, and B_{j+1} gathers every transition that runs while B_j is written. T denotes $\hat{T}$ at the end of the last batch whose* put^* *returned* Ok*.*

Law 11 (Group commit)**.** *1)* Release. *r_i is returned only after $\mathrm{put}^*(W_{j(i)}) = \mathsf{Ok}$, whatever r_i is (a refusal, or a transition that writes nothing): a result never rests on a state that is not durable.*

2) Atomicity per record. put^* *discharges A per name: $\forall \sigma' \in R_P.\ \forall n \in \mathrm{dom}\, W_j.\ \sigma'(n) \in \{\sigma(n), W_j(n)\}$, with no order promised across names.*

3) Order. *For every name, the values* put^* *makes durable follow lock order: batches are written in order, one at a time, and $W_j(n)$ is the last write to n in B_j.*

4) Fail closed. *$\mathrm{put}^*(W_j) = \mathsf{Err} \Rightarrow$ every pending transition (those of B_j and of every later batch, all computed on $\hat{T}$ past T) returns* Err*, and $\hat{T} \leftarrow u_{\mathrm{first}} \circ \cdots \circ u_{\mathrm{last}}(\hat{T}) = T$: no member is adopted. A failed* put^* *may have landed some of W_j (atomicity is per record), so memory $\sqsubseteq$ disk, and every member durable but undone is a lost release (Prop. 9).*

5) Latency. *With ϕ a bound on one* put^**, a transition is released at most 2ϕ after it ran: the batch in flight, then its own.*

6) Size. *$|\mathrm{dom}\, W_j|$ is at most the store's record bound, and its bytes at most the store's byte bound, since W_j holds one record per name; $|B_j|$ is the number of transitions in flight, each held by its caller, as each was queued on the lock before.*

Native put^***.**

$$\forall n \in \mathrm{dom}\, W:\ \mathrm{write}(n.\texttt{tmp});\ \mathrm{push}(n.\texttt{tmp});\quad \mathrm{barrier};$$
$$\forall n:\ \mathrm{rename}(n.\texttt{tmp}, n);\quad \mathrm{sync}_{\mathrm{dir}};$$
$$[\,\forall n \notin P:\ \mathrm{create}(n.\texttt{p});\quad \mathrm{sync}_{\mathrm{dir}}\,]\quad \text{(iff some } n \notin P\text{)}$$

where push moves a file's data to the device and barrier makes every pushed byte durable. On Apple targets push = `fsync` and barrier = `F_FULLFSYNC` once, which flushes the whole drive cache [3]; elsewhere push = `fsync`, already durable, and the barrier is empty. A batch therefore pays two full flushes (three when it creates a marker), as one put did. A rename is flushed only after the barrier, so a name never holds a torn record, and a marker is created only after its rename is flushed, so Invariant 1 holds: A and W per name, as Law 3. A store that refuses (Law 8) checks the whole batch against its budget before its first write.

IndexedDB put^***.** One read-write transaction under Law 7 holding every put of W, after one budget check: all or nothing, which is stronger than A per name.

Proposition 9 (A lost release)**.** *A member whose record landed and that was never released (a crash after some records of W_j landed and before B_j was released, or a failed* put^* *that landed it) is a transition that committed and whose caller dropped the result: an admission is in $\mathcal{W}_s$ and not dispatched (its retransmission is a replay, and the message is lost, as for any drop after admission); a lease extension covers sequences nobody was issued, so $\mathit{issued} \le \hat{L} \le D$ holds with D the last write; a custody entry names a delivery still in the carrier, which the next pass requeues or the restriction drops, and a landed release retires early only what the application already took (the duplicate window of Law 10).*

Proof. By atomicity per record each name holds its value before B_j or $W_j(n)$, and $W_j(n)$ is $\hat{T}$ restricted to n after the last member that wrote n. No member was released (Release), so no caller acted on its result. □

Corollary 2 (Leases)**.** *The lease invariant of* `message::replay::lease` *becomes* $\mathrm{returned} \le D$*: a reservation is released only after the batch holding its own or an earlier lease extension is durable, while the speculative lease $\hat{L}$ may exceed D until then. A failed batch undoes $(\mathit{issued}, \hat{L})$ of members none of which was returned.*

Corollary 3 (Custody)**.** *$\mathcal{W}_s$ and $\mathcal{K}_s$ stay one record, and a batch writes one value per record, so a crash still cannot separate an admission from its custody entry, nor a release from its effect: Law 10 holds unchanged.*

X. Claimed durability

A store at (Dur°, m) satisfies Law 1 read with the claim, and violates it read with the ephemeral case. Which reading applies is fixed by whose state the store holds: refusing a node's own state refuses the node, while refusing state it holds for other nodes moves that state to another holder.

Law 12 (Own state on a claim)**.** *A store of the node's own state (replay, entry guards) may be opened at (Dur°, m) for every m. In an ephemeral context each session end is λ_{tot} of the store, the documented reset of Proposition 2.*

For replay at $(\mathsf{Dur}^\circ, \mathsf{Aut})$ the reset is per ephemeral session: a receiver may admit an unexpired transaction once more, which its expiry bounds, and a sender restarts its streams at sequence 0, which destinations see as Stale or Fork until it passes their retained high watermark. For entry guards it is one re-selection of the first hop.

Relay carriers are the other case. Most nodes are expected to be browsers (#1036), so carriers must rest on browser holders: moving custody past every browser (#1037) would concentrate it on the few native nodes. One Dur° holder loses its carriers at an ephemeral session end, and a persistent but closed tab holds them unreachably until it returns. Both are holder losses, so redundancy across holders replaces durability at one holder.

Law 13 (Redundant custody)**.** *A carrier is admissible at* $(\mathsf{Dur}^{\circ}, \mathsf{Ret})$ *iff it is held by* $r \geq 2$ *holders, the first* r *relay-capable nodes from its key's owner under the holder law of #1037, and every repair period* T *restores* r *copies from any copy that survives. A carrier at an established* $(\mathsf{Dur}, \mathsf{Ret})$ *is admissible at* $r = 1$*.*

Proposition 10 (Survival across holders)**.** *Under Law 13 a held element is lost only if every holder that held it is lost (it left, or its storage was lost) within one repair period.*

Proof. A repair pass that runs while one copy of an element survives restores r copies. Between two passes the copies decrease only by holder loss, so the element is lost only if all of its copies, at least r after the last pass, are lost before the next. □

Proposition 6 then holds with its premise, a carrier that survives, read as the condition of Proposition 10. Today $r = 1$ for every carrier, so a browser carrier is an inadmissible point, as relay carriers in the data store were before #1021; #777 implements the law: placement on the first r relay-capable nodes, acknowledgements (#1037) that count holders, and its composition with the custody set $\mathcal{K}_s$.

XI. Replicated custody

Law 13 fixes what must hold; this section fixes how, composed with the holder law of #1037, the owner query of #1045, the acknowledgements of #1037, and the custody set of Law 10. Holders are lost to churn and to ephemeral sessions; availability against Byzantine holders is not claimed, so a member's copy is trusted as #1037 trusts a relocating holder's.

Write $\mathrm{holds}(n)$ for n's declaration, $\mathrm{succ}(n)$ for its successor head, $\mathrm{owner}(k)$ for the n with $k \in (n, \mathrm{succ}(n)]$, and $p_j(n)$ for the j-th nearest node strictly behind n with holds.

Definition 11 (Members)**.** $\mathcal{M}_r(k)$ *is the first* r *nodes* n *of* $\mathrm{owner}(k), \mathrm{succ}(\mathrm{owner}(k)), \mathrm{succ}^2(\mathrm{owner}(k)), \ldots$ *with* $\mathrm{holds}(n)$*; its first,* $h_0(k)$*, is the holder of #1037. The* rank *of a member is* $\mathrm{rank}_k(n) = |\{m \in \mathcal{M}_r(k) : m \text{ before } n\}|$*, and its* next member $\mathrm{next}_k(n)$ *is the member of rank* $\mathrm{rank}_k(n) + 1$*.* r *is configurable, default* $r = 8$*, aimed at a held message's loss below* 10^{-3} *over its 24 h retention with 90% browser holders (#777); which session models it covers is stated with the departure classes below, and it is to be recalibrated on #773's churn simulator.*

Lemma 2 (Member range)**.** *For* n *with* $\mathrm{holds}(n)$*:* $n \in \mathcal{M}_r(k) \iff k \in (\mathrm{succ}(p_r(n)), \mathrm{succ}(n)]$*, and* $n = h_0(k) \iff k \in (\mathrm{succ}(p_1(n)), \mathrm{succ}(n)]$*.*

Proof. $n \in \mathcal{M}_r(k)$ iff fewer than r holders lie in $[\mathrm{owner}(k), n)$, iff $\mathrm{owner}(k) \in (p_r(n), n]$. The arcs owned by the nodes from $\mathrm{succ}(p_r(n))$ to n are $(\mathrm{succ}(p_r(n)), \mathrm{succ}(n)]$. At $r = 1$ this is the holder's range of #1037. □

Law 14 (Previous holders)**.** *Each repair pass every node* n *passes to* $\mathrm{succ}(n)$ *the list* $\pi(n) = [(n, \mathrm{succ}(n))] +\!\!+ \rho(n)[..r-1]$ *if* $\mathrm{holds}(n)$*, and* $\rho(n)$ *otherwise, where* $\rho(n)$ *is the list its predecessor last passed it; a node takes a list from its predecessor alone. A list that comes round the ring to* n *names every holder, and* n*'s search for members then continues round the ring.*

By the induction of #1037 along each storage-less run, every holder learns $p_1(n), \ldots, p_r(n)$ with their successor heads, so Lemma 2 is decided on local state.

A member decides its rank on local state too: its list names $p_1, \ldots, p_r$, so it counts the holders in $[\mathrm{owner}(k), n)$. It finds $\mathrm{next}_k(n)$ in its successor list, or, when no node of that list holds relay mail, by the owner query of #1045 for the key just past its list's last node, whose owner answers the first holder at or after it; an owner whose view names none answers the last node it places, and the query is asked again past it. So a member reaches the next member however long the run between them, and the successor list stays at its capacity of 3: with browsers holding relay mail (#1036) storage-less runs are short, and the query covers the rest.

Law 15 (Age gate)**.** *A node declares that it holds relay mail only once it has been online for* A*, at least four stabilization periods (default 2 min); until then it routes, and fetches its own mail from its holder (#1037), but is no member and takes no copy.*

Replica maintenance costs each node about $r\,B/(N\bar{L})$ bytes per unit time, B the held bytes of the overlay, N its nodes and $\bar{L}$ the mean member session [10]: at $r = 8$, 10^5 nodes and a fifth of them receiving a full carrier of 4 KiB elements, about 27 KiB/s per node at $\bar{L} = 15$ s against 0.7 KiB/s at 10 min, and a node that leaves within its first session pays its join copy before it is of use. The gate excludes such nodes, and it makes members length-biased, which is the session model the default r assumes.

A member caches $\mathrm{next}_k(n)$ for the range of keys whose next member it is, until its view changes: every key of the range has the same next member, so one owner query serves them all, and a member asks at most one per view change. The successor capacity is revisited only if measured runs routinely exceed it: raising it from 3 to 8 costs five more keep-alives a node, less than routed queries per hold.

Law 16 (Chain replication)**.** *A hold goes to* h_0*, under the writer authority of #1037.* h_0 *joins the element and forwards the hold to* $\mathrm{next}_k(h_0)$*, digest first: the bytes follow only when the next member lacks the element. A member of rank* j *that admits it (Law 17) forwards it to its next member while* $j < q(k)-1$*, and the member of rank* $q(k)-1$ *acknowledges it to the member it came from; every member acknowledges the member it came from once its own next member acknowledged.* h_0 *acknowledges the writer once its next member did, at once when* $q(k) = 1$*. Here* $q(k) = r$*, or the number of holders when a member's list came round the ring (Law 14) and names fewer. The writer keeps and resends the hold until acknowledged, as before.*

So an acknowledgement means that every one of the $q(k)$ members holds the element: a chain acknowledges only from its end, and no member acknowledges for another.

Law 17 (Replica admission)**.** *m admits a copy of the carrier at k from s iff $m \in \mathcal{M}_r(k)$ (Lemma 2), $\mathrm{holds}(s)$, and s is m's previous member, $p_1(m)$ inside the range, or its next member. Admitted elements pass the witness of the carrier as before; a copy is a join (S3).*

Law 18 (Chain repair)**.** *Chain order is ring order over the holders, so every carrier a member m keeps has the same chain neighbours, its previous holder $p_1(m)$ and its next holder $n_1(m)$, but at the ends of its range (rank 0 has no previous member, rank $r-1$ no next). A repair pass, periodic or event-driven, sends at most one message to each of the two: the digest set of every carrier m keeps whose chain includes that neighbour, the bytes following only for the carriers it lacks. A member whose carriers an admitted copy changed offers them on to its other neighbour at once, in the same bundled form. A keeper that is no member offers the carrier to the member it can name and deletes its copy only after a delete-capable acknowledgement from a member holding $\sqsupseteq$ its state (S1″, S2′ with member for holder).*

Law 19 (Event-driven repair)**.** *A member that loses a node of its view, by a closed connection or a failed liveness check, runs Law 18 at once, on its new view, when that node was $p_1(m)$ or $n_1(m)$: one bundled exchange with each replacement neighbour, whatever the number of carriers. The periodic pass of period T remains the backstop.*

Proposition 11 (Repair delay)**.** *After the loss of one member of k, every member of k holds a copy again within $D \le T_d + r\,\delta$, where T_d bounds the loss's detection (a connection close at once, a liveness failure within its check) and δ one chain hop (a send and a join, and the owner queries when the next member lies past the successor list); within T when the event is missed.*

Proof. The loss is detected by the lost member's chain neighbours within T_d, and each runs Law 18 at once. A copy that changes a member's carrier is offered on at once, so the hole closes along the chain in at most r hops. A missed event leaves the periodic pass. □

Departure classes. T_d depends on how a member leaves. A *graceful* departure closes its connections, so its neighbours detect it at once: $T_d \approx 0$, and $D_g = r\,\delta$, seconds. A *silent* one (a crash, a dropped network, a suspended tab) is detected only when its liveness check fails, after `PEER_LIVENESS_TIMEOUT_MS` of 45 s: $D_s \approx 45\,\mathrm{s} + r\,\delta$, about a minute. With σ the fraction of departures that are silent and $\varepsilon_r(D)$ the probability that all r members of a key are lost within one window D, a held message's loss over its retention is bounded, to first order, by the mixture

$$\varepsilon \;\le\; (1-\sigma)\,\varepsilon_r(D_g) + \sigma\,\varepsilon_r(D_s),$$

since an element's window opens with the loss of its first member, whose class sets D (a later member's class only shortens or lengthens the same window's repair). By the table of #777, $r = 8$ meets 10^{-3} for Weibull ($k = 0.5$) sessions of median 1 min at $D = 60$ s, so for any σ; for a median of 15 s it meets it only at $D = 10$ s, that is with $\sigma \approx 0$, and a silent fraction at that median needs $r \approx 14$. Two levers move σ and D_s instead of r: a leave on `pagehide` (#1057) makes a closing tab's departure graceful, and a shorter liveness timeout between chain neighbours (#1070), which are only $p_1(m)$ and $n_1(m)$, bounds D_s; with them $r = 8$ covers the 15 s median. The laws hold for every r; these figures only choose its default.

Proposition 12 (Acknowledged copies survive)**.** *In a window where the views agree on $\mathcal{M}_r$ (S4), an acknowledged element has $q(k)$ copies, and is lost only if every member holding it is lost within D.*

Proof. Law 16 gives $q(k)$ copies at the acknowledgement. Copies decrease only by a member's loss or by a deletion of Law 18, made by a non-member after a member holds its state, which leaves the members' copies as they were. By Proposition 11 a loss is repaired from any surviving copy within D, which is Proposition 10 with D for the repair period. □

Outside S4. An acknowledgement taken while views disagree still means $q(k)$ nodes hold the element, each of which admitted it as a member of k by its own view; it does not mean they are the members the views will agree on. No copy is deleted without a member's acknowledgement (S1″), so their number does not fall but by node loss, and once the views agree Law 18 moves the copies to the members. Until then the bound of Proposition 12 holds with D extended by the convergence of Proposition 13.

Proposition 13 (Convergence)**.** *After one membership change the lists of Law 14 settle within $r(L+1)$ passes, L the longest storage-less run.*

Proof. A list moves one hop per pass, and a change reaches the r-th holder after it through at most r holders and the runs before each, $r(L+1)$ hops; at $r = 1$ this is the bound of #1037. □

Removal, fetch, owner. The recipient removes at the member it fetched from, and tombstones spread by Law 18. Tombstones stay capped at `RELAY_INBOX_MAX_LEN`, so a lagging member can resurrect an element whose tombstone fell off the cap; its redelivery is refused by the recipient's window (Replay, or Stale below it) and retired, case (ii) of Proposition 6, except after a replay reset (Law 12), where its retention bounds it. A recipient without a relay store fetches from h_0 and then down the chain; an element is the same holder-signed bytes at every member, so the drain's handed-over set and $\mathcal{K}_s$ deduplicate copies, and Law 10 is unchanged. Only h_0's report restricts custody: a lagging member's copy may miss an element that still exists (#1059).

Store capacity. A node keeps about r times the carriers it kept as a sole holder, so the relay store's budget scales with r: a browser's from 1024 rows to $1024\,r$, a native node's from 64 MiB to $64\,r$ MiB, a cap on a store that evicts nothing, not space it takes. A member whose store is full still refuses a copy (N): it does not acknowledge, so the chain acknowledges no writer, which resends until the sender proof expires and then drops the hold with a warning, as a full store's refusal did before. So exhaustion never lowers $q(k)$; it refuses new mail. Its blast radius is the keys whose chain contains the full

member, those whose owner lies in $(p_r(m), m]$ (Lemma 2): about r holders' arcs refuse new mail while it is full, and every other key is unaffected. Isolating one origin's load from another's within a store is the per-origin share of #907 and #1060, not this law's.

Proposition 14 (Message budget)**.** *Per node and per repair pass: one list of Law 14 and at most two bundled repair messages, so at most three, whatever the carriers it keeps; plus $2r$ for each hold it relays, and one bundled exchange per change of p_1 or n_1.*

Proof. Laws 14, 18 and 19 as stated; a hold is one send to h_0, $r-1$ forwards, $r-1$ chain acknowledgements and one to the writer, and the owner queries of its hops are served by the cached next members. □

Costs. A hold costs $r-1$ forwarded copies and $r-1$ acknowledgements, and the writer's acknowledgement waits for the chain's round trip, about $2(q-1)\,\delta \le 2r\,\delta$. A member that leaves mid-chain stalls the acknowledgement until its loss is detected and the chain repaired, within $D + 2r\,\delta$ of its departure (D by its class); the writer keeps and resends the hold meanwhile, so the stall delays the acknowledgement and loses nothing. A pass offers each carrier's digest set to two chain neighbours in one message each and copies only on a difference. A list of Law 14 carries r pairs. A best-effort leave on departure (#1057) shortens T_d and is never needed for this section's laws.

XII. Implementation order

PR	Issue	Law
1	#923	Law 8 on `IdbStorage`; Cor. 1
2	#912	Law 7; SECURITY.md next to the native text
3	#915	Missing in `ScannedRecord`; Laws 3, 4, 5, 6
4	#911	`DurableCache`; Law 9
5	#936	Law 10; receiver record gains $\mathcal{K}_s$

The dependencies: 2 and 4 on the browser need the strict-commit seam of 2; 4 on Windows needs the write-through rename of 3; 5 needs nothing from 1–4, but its Proposition 6 holds in full only with #1021. PR 1 comes first because it is the only one that closes a safety gap reachable without a crash. #1021 and #1022 follow the batch; #777 implements Law 13 by Section XI.

Operator steps, in the CHANGELOG under `Unreleased`: PR 5 changes the receiver record, so an old record does not decode and fails its stream closed; the operator removes the `transaction-replay` store (native directory, browser database `<storage name>/transaction-replay`) while the node is stopped, accepting the reset of Proposition 2. PRs 1–4 need no step: an existing native store gains its markers at its first open, and the IndexedDB schema is unchanged.

XIII. Summary of laws

Predicate	Discharged by
presence, native	markers (Laws 3–5)
presence, browser	N and whole-origin loss (Prop. 3)
presence, total loss	undetectable; the documented reset
survival, unix	fsync, rename, directory fsync
survival, Windows	write-through rename (Law 6)
survival, browser	strict commit (Law 7)
presence of custody	$\mathcal{K}_s$ in the receiver record (Law 10)
own state, ephemeral	reset per session (Law 12)
custody, browser	r-member chain (Laws 13–19)
throughput, durable	group commit (Law 11)

References

[1] MDN Web Docs, "Storage quotas and eviction criteria," https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria, accessed 2026-10-23.

[2] W3C Web Applications Working Group, "Indexed database API 3.0," W3C Editor's Draft, https://w3c.github.io/IndexedDB/, section "Transaction durability"; accessed 2026-10-23.

[3] Apple, "`fsync(2)` and `fcntl(2): F_FULLFSYNC`," macOS manual pages, accessed 2026-10-23.

[4] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "All file systems are not created equal: On the complexity of crafting crash-consistent applications," in *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI '14)*. USENIX Association, 2014, pp. 433–448.

[5] Microsoft, "`MoveFileExW` function (`winbase.h`): `MOVEFILE_WRITE_THROUGH`," https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-movefileexw, accessed 2026-10-23.

[6] Chrome for Developers, "A change to the default durability mode in IndexedDB," https://developer.chrome.com/blog/indexeddb-durability-mode-now-defaults-to-relaxed, default changes from strict to relaxed in Chrome 121; accessed 2026-10-23.

[7] Mozilla, "IndexedDB parent actors (`ActorsParent.cpp`)," https://github.com/mozilla-firefox/firefox/blob/main/dom/indexedDB/ActorsParent.cpp, `BeginWriteTransaction` and `GetDefaultSynchronousMode`; accessed 2026-10-23.

[8] WebKit, "IndexedDB SQLite backing store (`SQLiteIDBBackingStore.cpp`)," https://github.com/WebKit/WebKit/blob/main/Source/WebCore/Modules/indexeddb/server/SQLiteIDBBackingStore.cpp, `commitTransaction`; accessed 2026-10-23.

[9] SQLite, "Write-ahead logging, and PRAGMA synchronous," https://www.sqlite.org/wal.html, https://www.sqlite.org/pragma.html, accessed 2026-10-23.

[10] C. Blake and R. Rodrigues, "High availability, scalable storage, dynamic peer networks: Pick two," in *Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, 2003, pp. 1–6.